Aplikace pro online doučování

Specifikace požadavků na systém

1. Funkční požadavky (Functional Requirements)

1.1 Správa uživatelů a přístupů

- **F-01: Obnovení smazaných/deaktivovaných účtů** – Systém musí umožnit administrátorovi vyhledat dříve deaktivovaného uživatele a zahájit proces obnovy jeho účtu.
- **F-02: Verifikace a notifikace obnovy** – Při obnově účtu musí systém vyžádat potvrzení akce, automaticky nastavit původní práva a odeslat uživateli informační e-mail o znovuzprovoznění účtu.
- **F-03: Diferenciace uživatelských rolí** – Systém musí rozlišovat mezi entitami Uživatel (Student), Lektor (Teacher) a Administrátor s různými úrovněmi oprávnění.

1.2 Systémová správa a údržba

- **F-04: Manuální zálohování (Dump)** – Systém musí umožnit administrátorovi ručně vyvolat kompletní zálohu (tzv. "Snapshot") databáze, souborů, zpráv a logů.
- **F-05: Zpětná vazba zálohování** – Po dokončení procesu zálohy musí systém vypsat stavovou zprávu o úspěchu/chybě a zapsat tuto informaci do systémového deníku.

1.3 Správa kurzů a obsahu

- **F-06: Moderace kurzů** – Administrátor musí mít nástroje pro úpravu (editaci) nebo úplné odstranění kurzů, které nevyhovují standardům.
- **F-07: Řízení viditelnosti** – V případě nahlášení nebo zjištění nevhodného obsahu musí systém umožnit okamžité stažení kurzu z veřejné nabídky bez nutnosti jeho okamžitého smazání.

---

2. Nefunkční požadavky (Non-functional Requirements)

- **N-01: Bezpečnost dat** – Veškerá komunikace při správě uživatelů a zálohování musí být šifrována pomocí protokolu HTTPS.
- **N-02: Auditovatelnost** – Každý zásah administrátora (obnova účtu, smazání kurzu) musí být zpětně dohledatelný v auditním logu (kdo, kdy a co změnil).

- **N-03: Dostupnost systému** – Proces zálohování nesmí způsobit výpadek systému pro běžné uživatele delší než 30 sekund.
- **N-04: Responzivita administrace** – Odezva systému na administrátorské úkony nesmí přesáhnout 2 sekundy (vyjma samotného procesu stahování dat při záloze).

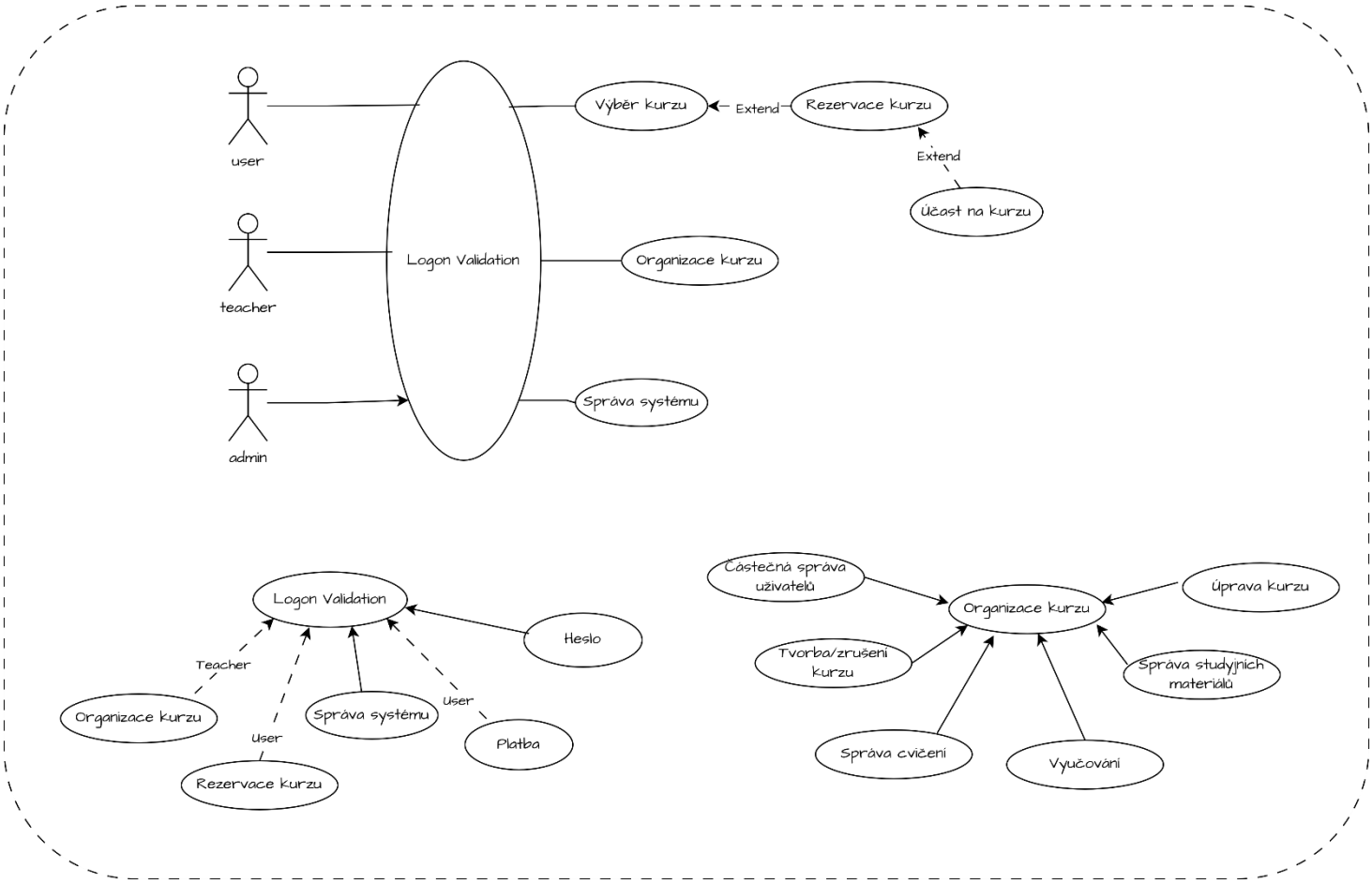
---

3. Akceptační kritéria (Scénáře užití)

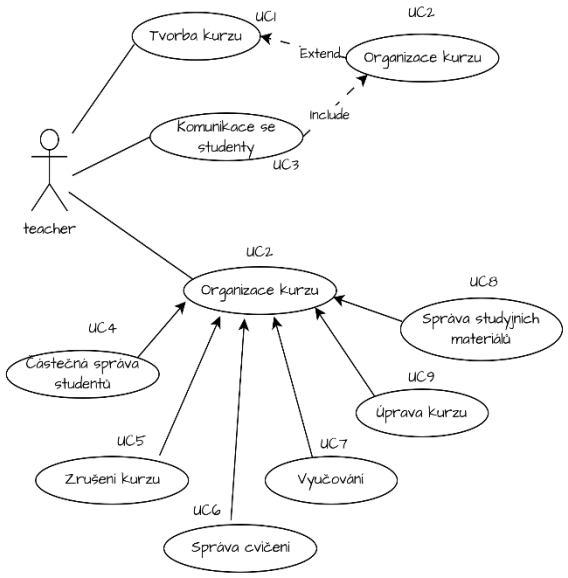
1. **Obnova účtu:** Scénář je úspěšný, pokud se uživatel po zásahu administrátora dokáže přihlásit se svými původními údaji a obdrží potvrzovací e-mail.
2. **Záloha:** Scénář je úspěšný, pokud je na serveru/cloudu vytvořen validní soubor obsahující aktuální stav databáze a administrátor obdrží potvrzení o dokončení.
3. **Smazání kurzu:** Scénář je úspěšný, pokud se kurz po smazání administrátorem přestane zobrazovat ve vyhledávání a v seznamu aktivních kurzů pro všechny uživatele.

Usecases

Základní struktura



Učitele



UC1

1) Učitel klikne na tlačítko vytvořit kurz

2) Systém zobrazí formulář pro vytvoření kurzu

3)Učitel správně vyplní formulář a klikne na vytvořit

4) Systém vytvoří kurz a přidá ho učiteli na účet

5) Dále učitel může upravovat daný kurz a spravovat ho, viz UC2

UC2

Systém umožní učiteli:

1) Přidání, úpravu a odstranění materiálů -> UC8

2) Přidání, úpravu a odstranění úkolů

3) Plánování a vytváření rozvrhu a jeho následná úprava

4) Zahajování a ukončení vyučování

5) Možnost smazání kurzu

UC3

1) Po zaplaceníkurzu uživatelem, se objeví možnost přímé komunikace se studenty (skupinově)

2) Po zaplaceníkurzu uživatelem, se objeví možnost přímé komunikace se studenty (skupinově)

UC4

1) Učitel má možnost nahlásit uživatele za nevhodné chování

2) Učitel má možnost ztlumovat uživatele v callu

UC5

1) Učitel má možnost zrušit a vymazat kurz

UC6

1) Možnost tvorby úkolů
2) Možnost úpravy úkolů
3) Možnost hodnocení, přidání komentářů u odevzdaných úkolů
4) Možnost zrušení úkolů

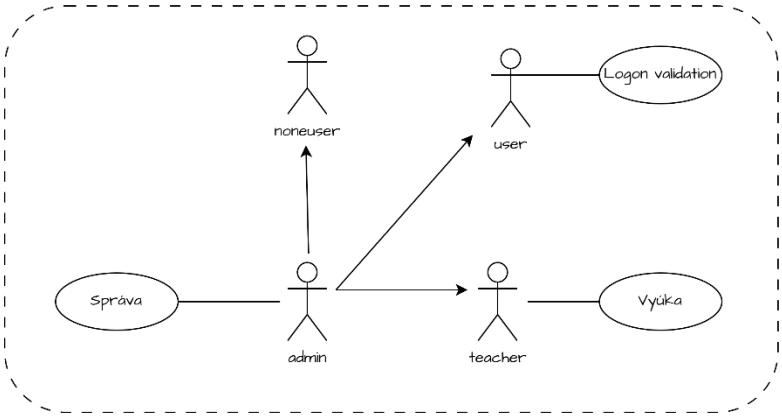
UC7

1) Učitel má možnost tvorby a zrušení hodin
2) Učitel může zahájit a ukončit hodinu
3) Možnost sdílení obrazovky, ztlumení uživatelů, atd.

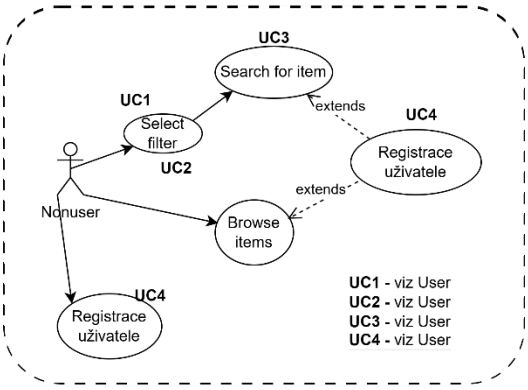
UC9

1) Změna informací o kurzu

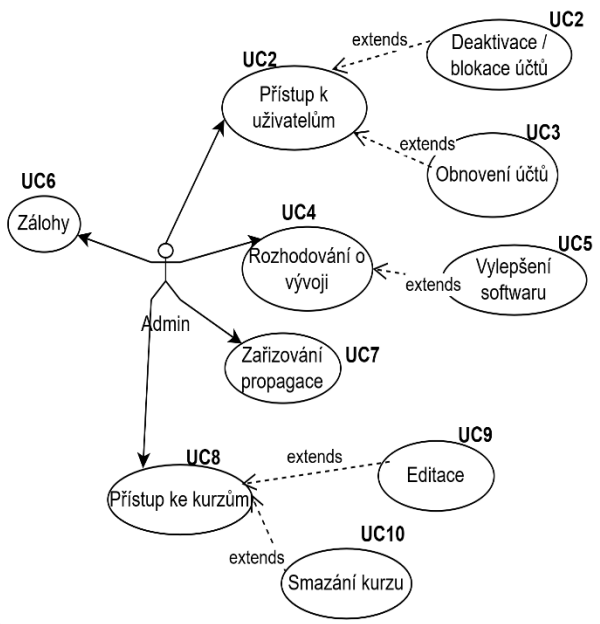
Actors



Nonuser



Admin

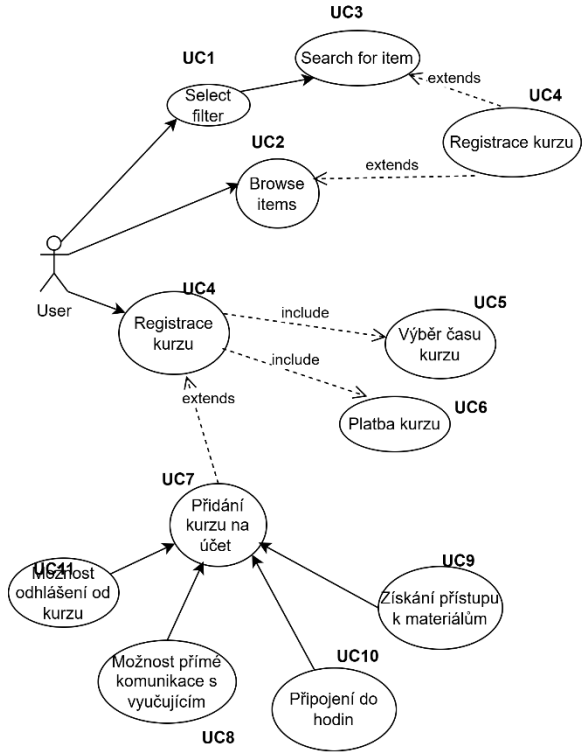


A1 - Obnovení dříve zablokovaného účtu
V kroku 4 administrátor vybere u již deaktivovaného uživatele možnost "Obnovení účtu"
Systém se zeptá na potvrzení akce
Po potvrzení systém uživateli opětovně přidělí práva, zaktivní jeho profil a zašle mu email, že jeho účet je opět plně funkční

A2 - Provádění systémových záloh (Zálohy)
V kroku 2 (na Dashboardu) administrátor nevyhledává uživatele, ale přejde do sekce "Údržba a zálohy"
Administrátor klikne na "Vytvořit novou zálohu"
Systém zahájí stahování / balení databáze (kurzy, účty, zprávy)
Po dokončení systém informuje administrátora o úspěšném vytvoření zálohy a zapíše časový údaj do logu

A3 - Smazání nebo úprava nevhodného kurzu
Pokud problém není v samotném účtu, ale jen ve vytvořeném kurzu, může administrátor v kroku 4 místo uživatele vybrat přímo konkrétní kurz
Administrátor kurz upraví (Editace) nebo jej úplně smaže, případně zruší z veřejného výpisu

Student



UC1
1) Zákazník vybere flitr vyhledávání
2) Systém zapne filtr pro vyhledávání

UC3
1) Uživatel zadá text do vyhledávacího pole
2) Systém nabídne nápovědu v textové formě k napsanému textu
3) Zákazník vyhledá věc
4) Systém vytvoří list vytvořených kurzů
5) Uživatel si vybere kurz
6) Systém nabídne detailní popis kurzu
7) UC4 - Registrace kurzu

UC2

UC4
1) Uživatel klikne na tlačítko registrace
2) Systém mu zobrazí registrační formulář
3) Uživatel vyplní info viz: UC5, UC6
4) Uživatel odešle formulář

UC5
1) Uživateli se zobrazí volné termíny
2) Uživatel si vybere volný termín

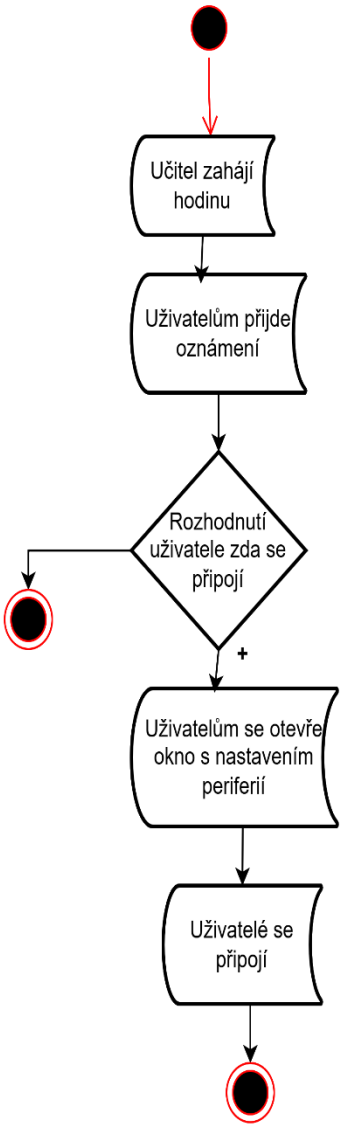
UC6
1) Systém odkáže na externí aplikaci
2) Systém ověří danou platbu

UC10
1) Systém odešle uživateli oznámení, že je hodina
2) Uživatel dostane možnost se zapojit do hodiny
3) Systém dá uživateli okno s možností nastavení periferií
4) Po připojení se uživatel může s hodinou interagovat

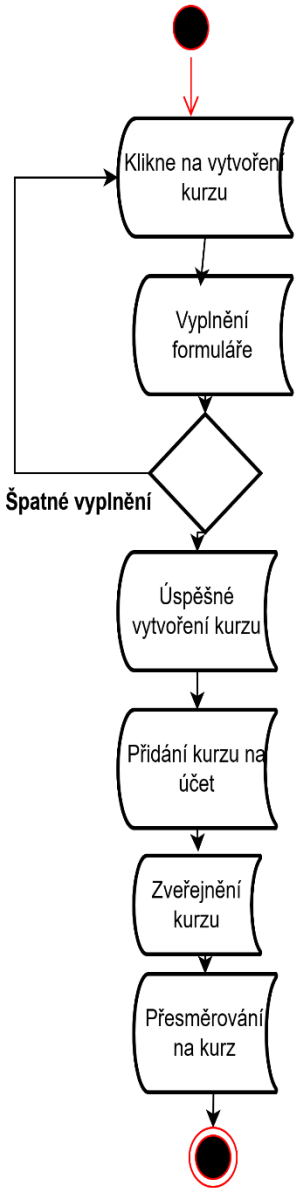
UC11
1) Systém umožní uživateli odhlášení a refundaci kurzu

Activity diagramy

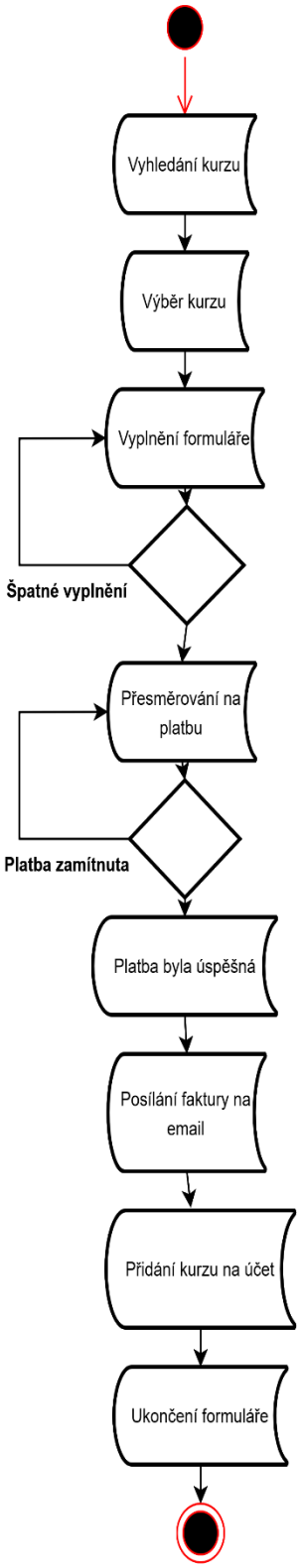
AD: Zahájení vyučování



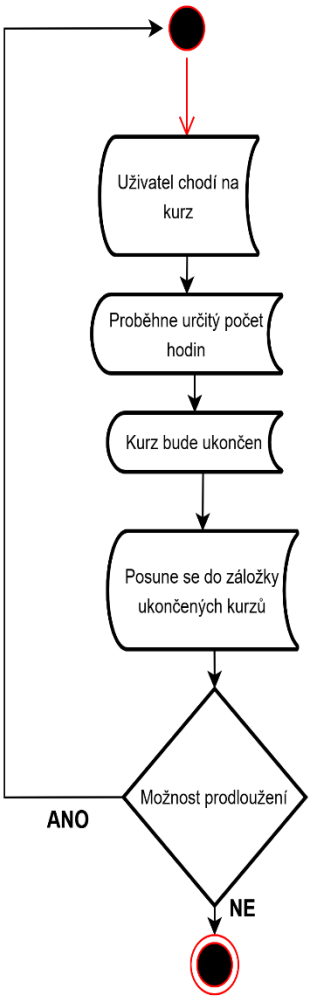
AD: Vytvoření kurzu



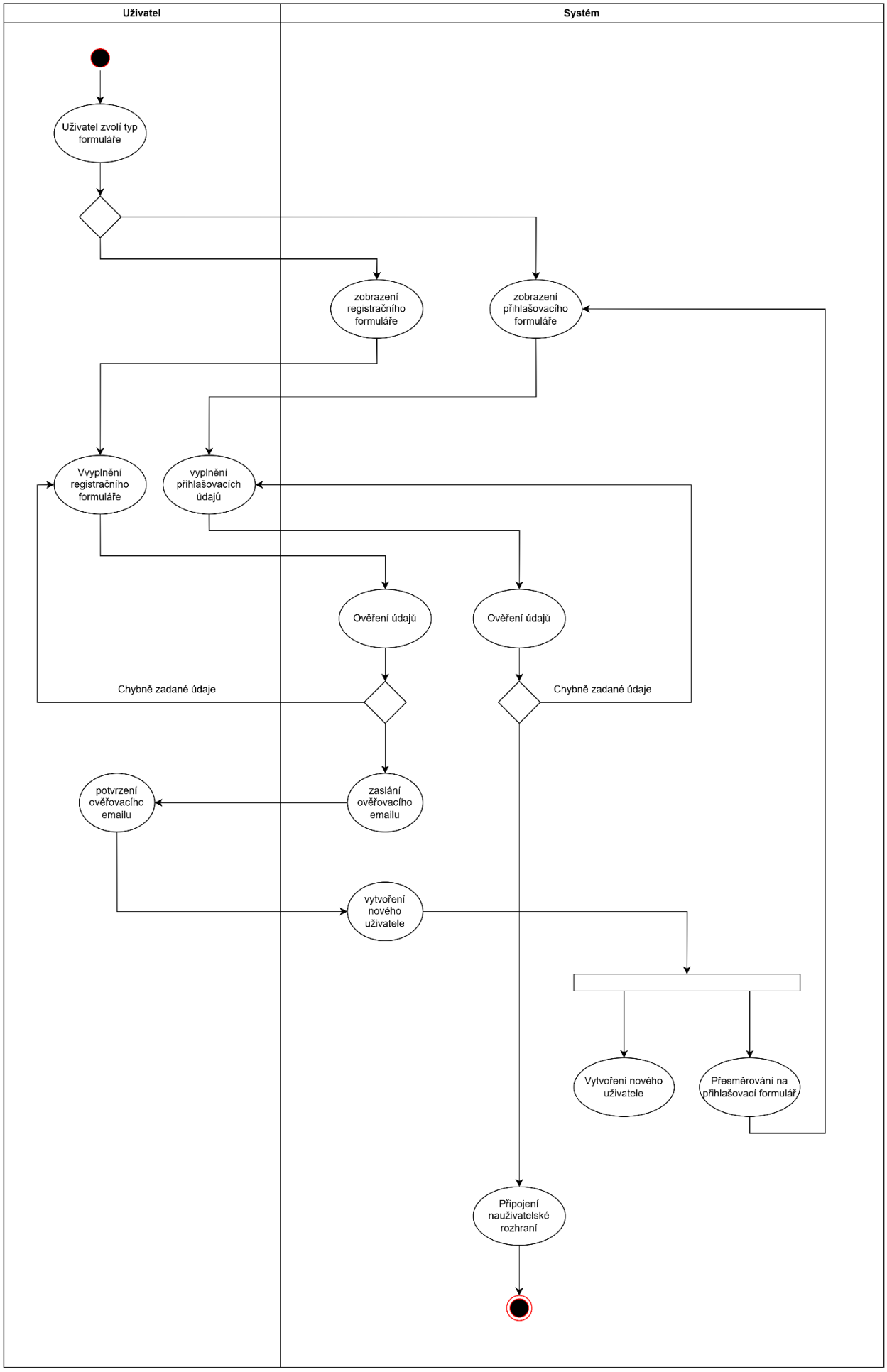
AD: Zakoupení kurzu



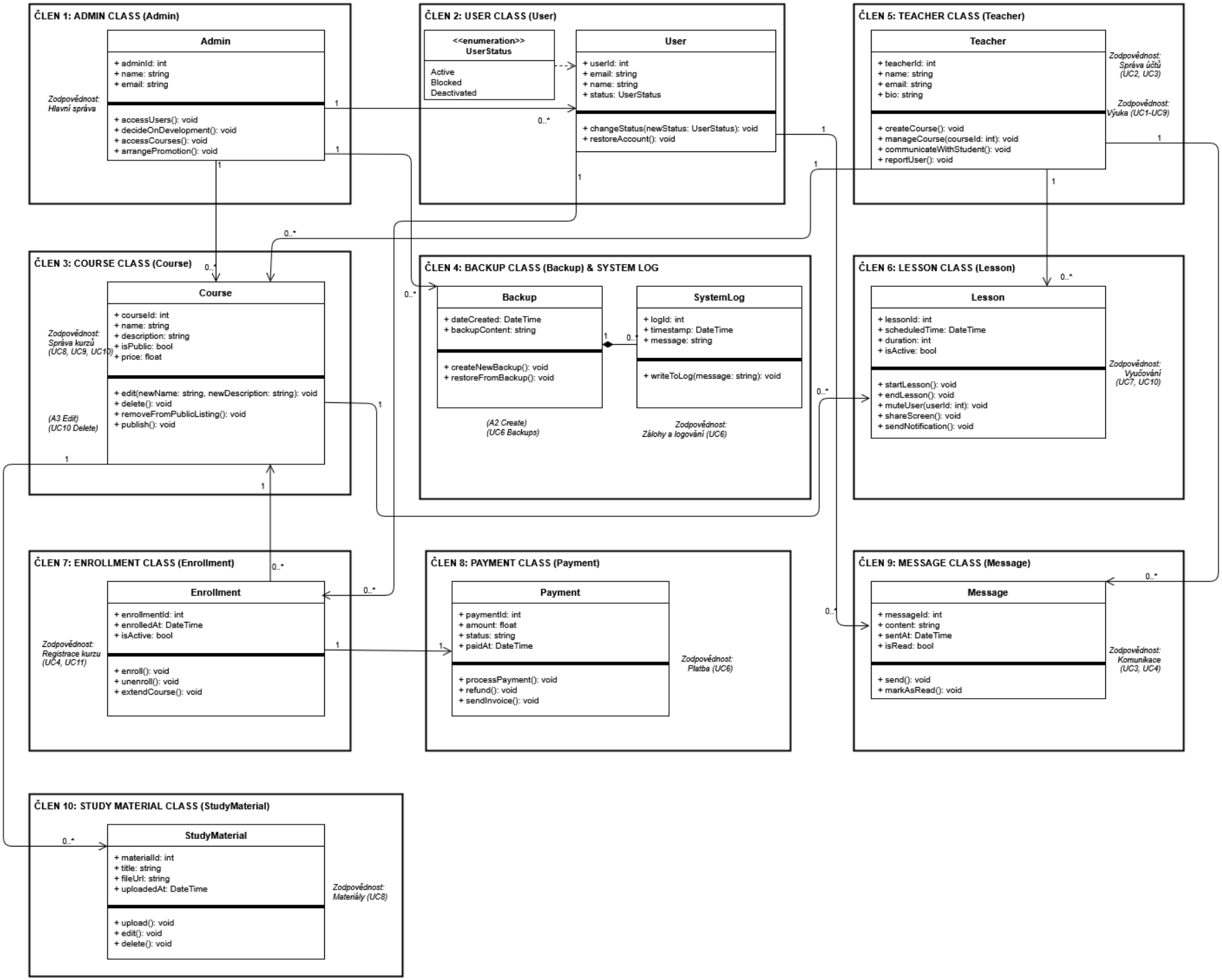
AD: Absolvování kurzu



Aktivitní diagram přihlášení a registrace



TŘÍDNÍ DIAGRAM - FULL MODEL (NÁVRH)



```cpp
// Knihovny
#include <iostream> // Pro C++
#include <stdlib.h>
#include <ctime>

using std::string;


class Admin{ // 1
private:
    int id_admin;
    string name;
    string email;
    string password;
    // ACCESS
    Teacher* accessTeacher;
    User* accessUser;

public:
    Admin(int id, string n, string e ,string p);
    ~Admin();

    void accessUser();
    void accessTeacher();
    void decideOnDevelopment();
    void accesssCourses();
    void arrangePromotion();
};

class Teacher{ // 2
private:
    int id_teacher;
    string name;
    string email;
    string bio;
    string password;
    Course* access;
    Comunication* chat;

public:
    Teacher(int id, string n, string e, string p, string b);
    ~Teacher();

    void createCourse();
    void manageCourse();
    void comunicateWithStudent();
    void reportUser();
};
```

```cpp
class UserStatus{ // 3
private:
    int status;

public:
    UserStatus(int s);
    ~UserStatus();
    string getStatus();
    void setStatus();
};


class User{ // 4
private:
    int id_user;
    string name;
    string email;
    string password;
    UserStatus* userStatus;
    Course* accessCourse;
    Comunication* chat;

public:
    User(int id, string n, string e, string p);
    ~User();

    void changeStatus(UserStatus newStatus);
    void restoreAccount();
    void comunicateWithTeacher();
    void courseAccess();
};

class Course{ // 5
private:
    int id_course;
    string name;
    string description;
    bool isPublic;
    float price;

public:
    Course(int id, string n, string des, bool stat, float pr);
    ~Course();

    void edit(string newName, string newDescription, float newPrice);
    void deleteSelf();
    void removeFromPublicList();
    void publish(bool stat);
};
```

```cpp
class Backup{ // 6
private:
    time_t timestamp;
    string backupContent;

public:
    Backup(time_t t, string bc);
    ~Backup();

    void createNewBackup();
    void restFromBackup();
};


class SystemLog{ // 7
private:
    int id_log;
    time_t timestamp;
    string message;

public:
    SystemLog(int id, time_t t, string ms);
    ~SystemLog();

    void writeToLog(string m);
    string getLog();
};
class Lesson{ // 8
private:
    int id_lesson;
    time_t scheduledTime;
    static int durationMin;
    bool isActive;
    User* users;
    Teacher* teacher;

public:
    Lesson(int id, time_t sT, int dur);
    ~Lesson();

    void startLeson();
    void endLeson();
    void muteUser(int userId);
    void shareScreen();
    void sendNotification();
};
```

```cpp
class Comunication{ // 9
private:
    Message* message;
    Teacher* teacher;
    User* user;

public:
    Comunication(Teacher* t, User* u);
    ~Comunication();
    void chat();
};


class Message{ // 10
private:
    int id_message;
    string content;
    time_t sentAt;
    bool isRead;

public:
    Message(int id, string c);
    ~Message();

    bool send();
    void markAsRead();
};
```